

# Audyt techniczny zdrovena-reconciliation

Plan refaktoru, lista usterek i mapa drogowa wdrożenia

|               |                                                      |
|---------------|------------------------------------------------------|
| Repozytorium  | github.com/PiotrGry/zdrovena-reconciliation          |
| Wersja        | v2.6.0                                               |
| Data audytu   | 19 czerwca 2026                                      |
| Zleceniodawca | Piotr Gryzlo · HUMIO sp. z o.o.                      |
| Zakres        | Backend, frontend, infrastruktura, testy, CI/CD      |
| Skala kodu    | ~21 000 LOC (11k backend, 7,6k testy, 2,7k frontend) |

Projekt jest na bardzo dojrzałym poziomie inżynierskim. Ma świadomą architekturę, agresywne quality gates, infrastrukturę jako kod, OIDC, automatyczny rollback i mutation testing. Większość uwag w tym raporcie to usprawnienia, nie blokery. Zidentyfikowano jednak 3 krytyczne usterki wymagające szybkiej reakcji oraz jeden problem architektoniczny blokujący rozwój produktu dla wielu użytkowników.

3  
Krytyczne

12  
Wysokie

8  
Średnie

~6  
Tygodni pracy

# 1. Streszczenie wykonawcze

Projekt zdrowena-reconciliation realizuje automatyzację zamknięcia miesiąca dla HUMIO sp. z o.o. — pobiera faktury z Fakturowni i KSeF, koreluje je, generuje paczki dla biura rachunkowego i wysyła powiadomienia. Jakość inżynierska jest ponadprzeciętna dla projektu wewnętrznego, ale produkt narósł kilkoma problemami, które obecnie utrudniają codzienne użytkowanie i blokują skalowanie na wielu użytkowników wewnątrz firmy.

## Najważniejsze wnioski

### Checkpoint nie działa zgodnie z intuicją

Mechanizm wznawiania pipeline po awarii zapisuje stan, ale nie weryfikuje obecności artefaktów. Po przerwaniu i restarcie kroki są pomijane mimo braku ich efektów. Dry-run i prawdziwy run zapisują pod różnymi kluczami, co dezorientuje system.

### Inbox niszczy pliki wejściowe

Przy każdym uruchomieniu pliki z katalogu inbox/ są przenoszone (nie kopiowane) do folderów docelowych. Restart po częściowej awarii wymaga ponownego ręcznego wgrywania plików — niezgodne z zasadą idempotentności.

### KSeF gubi faktury wystawiane przez księgowych

Zapytanie do KSeF używa tylko parametru `dateType=Issue`. Faktury wystawione w kolejnym miesiącu za okres zamykany (typowe dla biur rachunkowych) wypadają z okna lub są odrzucane przez filtr daty. Ponadto brak dopasowania w Fakturowni generuje tylko ostrzeżenie, nie blokuje wysyłki niekompletnej paczki.

### Pipeline trzymany w pojedynczym żądaniu HTTP POST

Odświeżenie przeglądarki gubi widok statusu, mimo że proces na serwerze działa dalej i kończy się wysyłką e-maila. Brakuje streamowania zdarzeń (SSE) i trwałego identyfikatora uruchomienia.

### Frontend pokazuje fałszywy postęp

Animacja kroków sterowana stałymi czasami (`STEP_EST_MS`) działa niezależnie od rzeczywistego przebiegu pipeline. Stąd subiektywne wrażenie „desynchronizacji konsoli z numeracją etapu”.

### Brak modelu wielu użytkowników w jednej firmie

Mimo że autoryzacja Entra ID i RBAC są skonfigurowane, brak blokady na poziomie uruchomienia — dwóch użytkowników może równolegle zamykać ten sam miesiąc, powodując uszkodzenie danych.

## Co działa wzorcowo

- Architektura warstwowa (`common` → `audit` → `month_closing` → `api`) egzekwowana fitness testami w CI.
- JWT z walidacją issuer + audience, RBAC z trzema rolami, OIDC zamiast service principal.
- Mutation testing w cron, property-based tests (Hypothesis), 35 plików testów, 80% coverage gate.
- Infrastruktura jako kod (Terraform), auto-rollback po smoke testach, SHA-pinning Actions.

- Świadomie udokumentowane decyzje techniczne (CLAUDE.md, TODOS.md, komentarze w omit).

## 2. Metryki projektu

| Obszar          | Liczby                        | Komentarz                            |
|-----------------|-------------------------------|--------------------------------------|
| Backend Python  | 11 075 LOC, 4 moduły          | Acyclic, fitness-tested              |
| Frontend React  | 2 713 LOC                     | Brak unit testów                     |
| Testy           | 7 743 LOC, 35 plików          | Property + fitness + mutation        |
| E2E             | 4 spec'i Playwright           | Testują endpointy, nie UI flow       |
| CI/CD           | 12 workflowów, 1 779 LOC YAML | OIDC, SHA-pinning, rollback          |
| Terraform       | ~1 400 LOC                    | Checkov, OIDC, RBAC managed identity |
| Coverage gate   | 80% globalnie                 | Bez per-module thresholds            |
| Coverage realne | ~82% mierzone                 | Krytyczne moduły w omit              |
| Pyright mode    | basic                         | Świadomy dług techniczny             |
| Sekrety         | Key Vault + DefaultCredential | Brak hardcode                        |
| Baza            | Azure Table Storage           | Single-tenant, OK skala              |

### Coverage w rozbiciu na moduły (estymata)

Z konfiguracji `[tool.coverage.run] omit` wynika, że krytyczne pliki nie są mierzone: `ksef.py`, `zoho_mail.py`, `fakturownia_reports.py`, `invoice_date_check.py`, `table_history.py` oraz wszystkie `commands/*.py`. Globalne 80% jest więc realne tylko dla utility code; ścieżki biznesowe (KSeF, mail, OCR PDF) są pokryte testami integracyjnymi/manualnymi.

### 3. Krytyczne usterki

Trzy problemy o wysokim ryzyku biznesowym — wymagają interwencji w ciągu 1-2 tygodni.

#### KRYTYCZNY

#### Shopify webhook akceptuje żądania bez walidacji HMAC

Plik `zdrovena/api/routers/webhooks.py:269-281`. Jeśli sekret `shopify_webhook_secret` nie jest skonfigurowany w Key Vault, walidacja HMAC jest pomijana z samym ostrzeżeniem w logu. Konsekwencja: błąd konfiguracji KV oznacza, że dowolny POST na endpoint webhook'a tworzy draft wysyłki w Twoim imieniu (InPost/Apaczka). Naprawa (15 minut): w trybie prod brak sekretu musi zwracać 503, nigdy 200.

#### KRYTYCZNY

#### Race condition przy zapisie draftów wysyłki

Plik `zdrovena/api/routers/webhooks.py:83-108`. Funkcje `_read_drafts` / `_append_draft` wykonują read-modify-write na pliku JSONL w blobie bez lease/ETag. Konsekwencja: dwa równoległe webhooks Shopify (typowe przy promocji/wyprzedaży, gdy Shopify retry'uje) prowadzą do silent loss draftu — jeden zostaje nadpisany. Naprawa: użyć blob lease (acquire/release) lub If-Match z retry; alternatywnie rozdzielić każdy draft na osobny blob i agregować przy odczycie.

#### KRYTYCZNY

#### KSeF nie znajduje faktur od księgowej (i pipeline o tym nie informuje)

Plik `zdrovena/month_closing/ksef.py:209`. Zapytanie używa `dateType="Issue"`, czyli filtruje po dacie wystawienia. Faktury wystawione w czerwcu za maj (typowe dla biur rachunkowych) wypadają z okna lub są odrzucane filtrem `sell_date/issue_date` w `orchestrator.py:482-488`. Dodatkowo: brak dopasowania w Fakturowni generuje tylko warning (`orchestrator.py:464-479`) — pipeline kontynuuje, ZIP idzie do księgowej z brakującymi fakturami. Naprawa: dual query (Issue + PermanentStorage), gate eskalowany z warning na blocker, filtr daty z fallbackiem dla brakującego `sell_date`.

## 4. Usterki wysokie

### WYSOKI

#### Checkpoint nie weryfikuje artefaktów

Plik `month_closing/state.py`. `PipelineState.is_done(step)` sprawdza tylko flagę w JSON, nie weryfikuje, czy efekty kroku (pliki PDF, ZIP, drafty) istnieją w blobach. Jeśli między uruchomieniami plik został usunięty, pipeline pomija krok i leci dalej z niekompletnymi danymi. Plus: dry-run zapisuje pod kluczem z suffixem (dry-run), prawdziwy run pod kluczem bez suffixu — frontend ma `normalizeStepKey()`, backend nie ma.

### WYSOKI

#### Inbox z destruktywnym `shutil.move`

`orchestrator.py:401-407` przenosi pliki z `inbox/` do docelowych folderów `shutil.move()`. Po częściowej awarii użytkownik musi wgrywać te same pliki ponownie. Standardowy wzorzec dla idempotentnych pipeline'ów: kopiowanie do destynacji + przenoszenie do `inbox/_processed/` dopiero po pełnym sukcesie.

### WYSOKI

#### Pipeline w jednym żądaniu HTTP POST

`POST /api/close` trzyma otwartą sesję HTTP przez 30-90 sekund. Odświeżenie karty gubi widok, ale proces na serwerze działa dalej (potwierdzone przez maile, które przychodzą po refresh'u). Brak `request.is_disconnected()`, brak `run_id`, brak SSE/WebSocket.

### WYSOKI

#### Fałszywy progres na froncie

`RunPanel.jsx:6` `STEP_EST_MS = [2000, 1000, 5000, 8000, 12000, 2000, 4000, 3000]` — animacja sterowana `setTimeout`. Jeśli backend trwa dłużej niż estymata, UI pokazuje „done” zanim cokolwiek się zakończyło. Dokładnie ten mechanizm powoduje subiektywne wrażenie desynchronizacji.

### WYSOKI

#### God class `orchestrator.py` (873 linii)

Cztery prawie identyczne `execute / execute_zip_only / execute_send_only / execute_zip_and_send`. Każda metoda `_step_X` robi własne IO, KSeF, Fakturownia, PDF processing. Refactor na listę kroków (Step pattern) umożliwi: per-step UI, mutation testing krok po kroku, równoległą realizację kroków niezależnych.

### WYSOKI

#### Brak blokady wielu użytkowników

Brak `started_by` w stanie pipeline, brak locku per (year, month). Dwóch użytkowników (Ty + księgowa, Ty + wspólnik) może równolegle zamknąć maj 2026, modyfikując te same bloby. Wynik: nieprzewidywalny chaos. Naprawa: tabela `locks` w Azure Tables z optimistic concurrency (ETag-If-Match) i auto-expiry.

### WYSOKI

#### Duplikat stanu dry-run na froncie

`CloseView.jsx:34, :44` trzyma dwa stany: `dryRun` (checkbox) i `runDryRun` (dla bieżącego runu). Plus przycisk „Sprawdź (dry-run)” nadpisuje checkbox bez zmiany jego stanu. Sprzeczne modele mentalne UX, mylące dla użytkowników innych niż autor.

WYSOKI

### Brak unit testów frontendu

Frontend (2713 LOC) ma tylko 4 spec'i E2E Playwright, zero vitest/jest. Krytyczne pure functions (normalizeStepKey, formatters, markRunningAsError) nie mają sieci bezpieczeństwa. Każdy refaktor frontu odbywa się obecnie ślepo.

WYSOKI

### CORS otwarte na localhost w produkcji

api/main.py: gdy brak ALLOWED\_ORIGINS, fallback to http://localhost:5173 z allow\_credentials=True. Niedokończony deployment = otwarcie brzegu. Trzeba fail-fast przy starcie.

WYSOKI

### Bug w \_get\_sender — błędne pole imienia/nazwiska

webhooks.py:53-55: lastname=sender\_name, firstname="". Pełne imię i nazwisko z Shopify trafia do pola lastname InPost/Apaczka. Działa technicznie (paczkomat patrzy na całość), ale brudne dane w audycie.

WYSOKI

### Mylący gate „warnings” (semantyka różna od nazwy)

orchestrator.\_check\_warnings\_gate sugeruje, że ostrzeżenia blokują pipeline. W rzeczywistości blokują tylko email, ZIP idzie dalej. Zmienić nazwę na \_check\_email\_gate\_warnings albo zmienić semantykę.

WYSOKI

### Rozjazd wersji README vs pyproject

README mówi 2.0.0, pyproject 2.6.0. Brak automatyzacji bumpowania wersji (release-please, semantic-release). Drobiazg, ale dla projektu z mutation testingiem i pełnym CI to uderzający w spójność.

## 5. Usterki średnie i niskie

ŚREDNI

Pyright w trybie basic, nie strict

Świadomy dług udokumentowany w CLAUDE.md. Plan: nowe moduły z `# pyright: strict`, stopniowo flip default gdy 80% modułów na strict.

ŚREDNI

time.sleep blokuje FastAPI worker

client.py:253, 305 używa `time.sleep(self.pdf_delay)`. Nieszkodliwe w CLI, ale blokuje worker FastAPI przy 1-2 replikach.

ŚREDNI

Brak unit testów dla scenariuszy resume checkpointu

test\_state.py testuje PipelineState izolowanie. Brak scenariuszy: „padł na kroku 4, restart, sprawdź pomijanie 1-3”, „artefakt z kroku 2 usunięty, sprawdź re-run kroku 2”, „dry-run nie miesza z prawdziwym”.

ŚREDNI

E2E testują endpointy, nie UI flow

close-flow.spec.ts wywołuje `POST /api/close` bezpośrednio, nie testuje interakcji użytkownika z CloseView. Brak testu „odśwież w trakcie i sprawdź wznowienie”.

ŚREDNI

Mutation testing pokrywa tylko 2 moduły

mutation.yml: sections.py + client.py. To ~5% kodu. Po refaktorze orchestratora dorzucić state.py, preflight.py, auth.py, webhooks.py.

ŚREDNI

Konsola źle synchronizowana z numeracją etapu

console.py:27 `step(self, n, title, total=7)`, a orchestrator ma 8 kroków numerowanych 0-7. Frontend ma 1-8. Wszystkie 3 numeracje są różne — stąd „desynchronizacja stdout z numeracją etapu”.

ŚREDNI

FilesView.jsx (561 linii) — god component

Powinien być rozbity na hooki (`useFiles`, `useUploads`) i pod-komponenty (`FileRow`, `UploadDropzone`). Bez tego każda zmiana w widoku plików ryzykuje regresję w wielu miejscach.

ŚREDNI

Frontend data.js miesza config z fixturami demo

KPIS\_STUB, SALES\_INVOICES, PRODUCTS, ORDERS lecą do prod-bundla obok prawdziwego PIPELINE\_STEPS. Rozdzielić na config/format/fixtures, fixtures gated przez vite define.

ŚREDNI

12 useState w CloseView

Stan rozproszony, łatwo o niespójność. Wyciągnąć logikę do hook'a `useCloseRun()`, ewentualnie `useReducer` z akcjami `START_RUN`, `STEP_DONE`, `RUN_COMPLETED`.



NISKI

console.warn w auth.jsx (info-leakage)

Diagnostyczne logi MSAL widoczne w DevTools użytkownika końcowego. Gated przez `import.meta.env.DEV` lub debug flag.

NISKI

Coverage fail-under tylko w CI, nie w pyproject

Dev odpalający lokalnie pytest nie wie o spadku coverage. Dodać `[tool.coverage.report] fail_under = 80`.

NISKI

Brak per-module coverage thresholds

80% globalnie nie pokazuje, gdzie konkretnie testów brakuje. Dodać per-area gate (audit  $\geq 85\%$ , common  $\geq 90\%$ , month\_closing  $\geq 70\%$ ).

NISKI

Brak SBOM/cosign w pipeline buildu

Dla wewnętrznego tool'a overkill, ale skoro masz OIDC, SHA-pinning, trivy, checkov — cyclonedx-bom + cosign dokończą obraz.

NISKI

Brak ephemeral environment per PR

Każdy PR deploy'uje do wspólnego staging. Concurrency lock kolejkuje. Przy 2+ devach warto rozważyć preview environments (Container App revision suffix).

NISKI

Brak performance tests w CI

Tylko correctness. Nie wykryjesz regresji typu „dry-run trwał 30s, po refaktorze 90s”.

## 6. Architektura — decyzje strategiczne

### Kontekst

Projekt został zaprojektowany jako single-user (Piotr) z infrastrukturą single-tenant. Obecnie planowane jest wprowadzenie kilku użytkowników w ramach HUMIO (wspólnicy, księgowia). Nie jest planowane multi-tenant SaaS — tool zostaje wewnętrzny.

### Decyzje, które potwierdzamy

- Pozostać przy single-tenant deployment. Brak migracji do multi-tenant SaaS, brak Stripe/billingu, brak onboarding wizard.
- Azure Table Storage jako baza state'u. Już skonfigurowana (RBAC, RM ID, DefaultAzureCredential), wykorzystanie minimalne. Wystarcza dla single-tenant z kilkoma userami.
- Entra ID single-tenant. Bez przejścia na Clerk/Auth0, bez Entra multi-tenant. Bo użytkownicy są pracownikami HUMIO, mają konta w tym samym tenant'cie.
- FastAPI + Container Apps + React + Vite. Stack zostaje, bez przepisywania.
- JSONL + Blob Storage do plików. Faktury, ZIP, drafty — dalej w blobie. State i lock w Tables.

### Nowe decyzje architektoniczne

#### Refaktor pipeline na Step pattern

`Orchestrator.execute_*`() → `Orchestrator.execute(steps: List[Step])`. Każdy krok jako klasa z metodą `run(ctx, run_id) -> StepResult`. Step jest idempotentny (umie sprawdzić, czy artefakty istnieją) i ma metodę `verify()` dla checkpointu.

Konsekwencje: per-step UI (jedna lista, każdy krok osobno), mutation testing krok po kroku, manual mode (odpalanie pojedynczych kroków) bez duplikacji kodu, jednolita numeracja kroków.

#### Server-Sent Events zamiast fake-progress

`GET /api/close/runs/{run_id}/stream` jako `StreamingResponse(media_type="text/event-stream")`. Backend emituje zdarzenia typu `step/log/done`. Frontend `EventSource` reconnectuje automatycznie po `refresh'u` (`Last-Event-ID`).

Eliminuje: `STEP_EST_MS`, `animate()`, `animStoppedRef`, bufor `StringIO` w odpowiedzi `POST`. Rozwiązuje problem desynchronizacji oraz częściowo problem `refresh'u`.

#### Lock per (year, month) na Azure Tables

Tabela `locks` z `PartitionKey="lock"`, `RowKey="{year}-{month}"`. Optimistic concurrency via `Etag-If-Match`. Auto-expiry po 1h. Pipeline odświeża lock co 30s w tle.

#### Nowe tabele state'u: runs + run\_steps

Zamiast pojedynczego JSON checkpoint blob → znormalizowane tabele: `runs` (jeden wiersz na uruchomienie, z `run_id`, `started_by`, `current_step`, `status`), `run_steps` (per-step status, artefakty, warnings). Frontend pobiera `GET /close/runs/{run_id}` z pełnym widokiem wszystkich kroków.

#### Niedestruktywny inbox z \_processed/

inbox/file.pdf → kopiuj do {year}/{month}/koszty/file.pdf, po sukcesie krok przenosi  
inbox/file.pdf → inbox/\_processed/{run\_id}/file.pdf. Re-run czyta z inboxu (jeśli nie został  
przeniesiony) lub z \_processed (jeśli już został). Pełna idempotencja.

## 7. Plan wdrożenia

Realizacja w trzech fazach. Faza 1 to absolutne minimum, żeby projekt nadawał się do codziennego użytku. Faza 2 otwiera narzędzie na innych użytkowników w HUMIO. Faza 3 to inwestycja w długoterminową utrzymywalność.

| Faza            | Cel                                                                                                        | Czas     | Status końcowy                                                          |
|-----------------|------------------------------------------------------------------------------------------------------------|----------|-------------------------------------------------------------------------|
| 1. Stabilizacja | Naprawić krytyczne usterki, refactor orchestrator na Step pattern, SSE, per-step UI, idempotentny inbox.   | 2-3 tyg. | Solidny single-user tool dla Piotra. Brak "checklamiącego" checkpointu. |
| 2. Multi-user   | Lock per (year, month), audit log, config UI dla vendorów, UX dla nie-Piotra, powiadomienia dla księgowej. | 1-2 tyg. | Tool gotowy dla współnika i księgowej. Brak race conditions.            |
| 3. Higiena      | Per-module coverage, vitest, mutation na więcej modułów, release-please, ephemeral environments per PR.    | 1 tydz.  | Pełna sieć bezpieczeństwa, automatyczny versioning, lepsze CI.          |

### Faza 1 — stabilizacja (2-3 tygodnie)

| #    | Zadanie                                                             | Czas  | Priorytet |
|------|---------------------------------------------------------------------|-------|-----------|
| 1.1  | Shopify HMAC fail-closed w prod                                     | 0.5d  | KRYT      |
| 1.2  | Fix CORS fail-fast gdy brak ALLOWED_ORIGINS                         | 0.5d  | KRYT      |
| 1.3  | KSeF: dual dateType query (Issue + PermanentStorage)                | 1d    | KRYT      |
| 1.4  | KSeF: gate eskalowany z warning na blocker                          | 0.5d  | KRYT      |
| 1.5  | Webhook Shopify: lease/ETag dla _read_drafts/_append_draft          | 1d    | KRYT      |
| 1.6  | Webhook: kolejka Service Bus + idempotent worker (lub queue Tables) | 2d    | WYS       |
| 1.7  | Refactor orchestrator.execute_* → Step pattern                      | 2d    | WYS       |
| 1.8  | Idempotentny inbox: kopiuje + _processed/                           | 1d    | WYS       |
| 1.9  | Checkpoint: weryfikacja artefaktów (Step.verify())                  | 1d    | WYS       |
| 1.10 | SSE endpoint /close/runs/{run_id}/stream                            | 1d    | WYS       |
| 1.11 | Per-step UI: lista kroków + indywidualne retry                      | 2d    | WYS       |
| 1.12 | Tryb expert: per-step run buttons                                   | 0.5d  | WYS       |
| 1.13 | Usuń STEP_EST_MS, animate(), runDryRun duplikat                     | 0.5d  | WYS       |
| 1.14 | Ujednolicić numerację kroków (1-indexed wszędzie)                   | 0.5d  | ŚR        |
| 1.15 | Fix _get_sender (firstname/lastname)                                | 0.5d  | ŚR        |
| 1.16 | Zmienić nazwę _check_warnings_gate                                  | 0.25d | NIS       |
| 1.17 | Dodać vitest + 10 unit testów frontendu (przed refaktorem)          | 1d    | WYS       |
| 1.18 | Dodać testy resume scenarios checkpoint (przed refaktorem)          | 1d    | WYS       |

|      |                                               |      |    |
|------|-----------------------------------------------|------|----|
| 1.19 | Integration test KSeF z VCR (po fix dateType) | 0.5d | ŚR |
|------|-----------------------------------------------|------|----|

Suma fazy 1: ok. 17 dni roboczych pracy programistycznej (2,5-3 tygodnie z testami i review).

## Faza 2 — multi-user w HUMIO (1-2 tygodnie)

| #    | Zadanie                                                     | Czas | Priorytet |
|------|-------------------------------------------------------------|------|-----------|
| 2.1  | Tabela locks na Azure Tables (optimistic concurrency)       | 1d   | WYS       |
| 2.2  | Tabela runs + run_steps (zastępuje JSON checkpoint)         | 2d   | WYS       |
| 2.3  | Endpoint GET /close/runs (historia z user info)             | 0.5d | WYS       |
| 2.4  | Endpoint GET /close/running (kto aktualnie zamyka)          | 0.5d | WYS       |
| 2.5  | Tabela vendor_config (vendorzy z UI)                        | 1d   | ŚR        |
| 2.6  | Settings UI: vendor config, always_ignore, email recipients | 1d   | ŚR        |
| 2.7  | Audit log: started_by w każdym uruchomieniu                 | 0.5d | WYS       |
| 2.8  | UX dla nie-Piotra: tooltips, mniej żargonu, błędy po polsku | 2d   | WYS       |
| 2.9  | Onboarding screen przy pierwszym wejściu nowego usera       | 0.5d | ŚR        |
| 2.10 | Email „paczka gotowa” do księgowej (SAS URL na 7 dni)       | 0.5d | WYS       |
| 2.11 | Email „pipeline padł” do admina                             | 0.5d | ŚR        |
| 2.12 | UI: banner „lock aktywny, X zamyka ten miesiąc”             | 0.5d | WYS       |
| 2.13 | E2E test: dwa równoległe sesje → drugi widzi lock           | 0.5d | ŚR        |

Suma fazy 2: ok. 11 dni roboczych (1,5-2 tygodnie z testami).

## Faza 3 — higiena długoterminowa (1 tydzień)

| #    | Zadanie                                                              | Czas  | Priorytet |
|------|----------------------------------------------------------------------|-------|-----------|
| 3.1  | Per-module coverage thresholds (audit ≥85%, common ≥90%, ...)        | 0.5d  | ŚR        |
| 3.2  | Coverage fail_under w pyproject.toml (nie tylko CI)                  | 0.25d | NIS       |
| 3.3  | Mutation testing: dodać state.py, preflight.py, auth.py, webhooks.py | 0.5d  | ŚR        |
| 3.4  | Soft mutation gate w PR (na zmienionych plikach)                     | 0.5d  | ŚR        |
| 3.5  | release-please-action: auto-bump wersji + CHANGELOG                  | 0.5d  | ŚR        |
| 3.6  | Plan przejścia pyright basic → strict (per-module)                   | 0.5d  | NIS       |
| 3.7  | Property tests dla PipelineState, Storage Protocol, HMAC             | 1d    | ŚR        |
| 3.8  | E2E: pełny UI flow (CloseView → progress → result)                   | 1d    | ŚR        |
| 3.9  | E2E: scenariusz refresh w trakcie pipeline                           | 0.5d  | ŚR        |
| 3.10 | Rozdzielić data.js: config / format / fixtures                       | 0.5d  | NIS       |
| 3.11 | Rozbić FilesView.jsx na hooki + pod-komponenty                       | 0.5d  | NIS       |
| 3.12 | Wyciągnąć useCloseRun() hook z CloseView                             | 0.5d  | NIS       |
| 3.13 | Wycofać duplikat lang.js (i18n on/off, nie pół na pół)               | 0.25d | NIS       |
| 3.14 | Gated console.warn w auth.jsx                                        | 0.25d | NIS       |
| 3.15 | Performance testy: track p95 latency dry-run close                   | 0.5d  | NIS       |
| 3.16 | Synchronizacja wersji README ↔ pyproject (release-please rozwiąże)   | 0d    | NIS       |

Suma fazy 3: ok. 7,75 dni roboczych (1 tydzień).

## 8. Sekwencja wykonania — tydzień po tygodniu

Kolejność jest istotna. Niektóre zadania muszą być zrobione przed innymi, bo stanowią sieć bezpieczeństwa dla późniejszego refaktoru.

### Tydzień 1 — security + przygotowanie

- 1.1 Shopify HMAC fail-closed (pierwsze, żeby zamknąć krytyczny wektor)
- 1.2 CORS fail-fast
- 1.5 Lease/ETag dla draftów Shopify
- 1.17 Vitest + 10 unit testów frontendu (SIEĆ BEZPIECZEŃSTWA przed refaktorem)
- 1.18 Testy resume scenarios checkpoint (SIEĆ BEZPIECZEŃSTWA przed refaktorem)

### Tydzień 2 — backend refactor

- 1.7 Refactor orchestrator → Step pattern (główne zadanie tygodnia)
- 1.9 Checkpoint z verify()
- 1.8 Idempotentny inbox + \_processed/
- 1.15 Fix \_get\_sender
- 1.16 Rename \_check\_warnings\_gate

### Tydzień 3 — KSeF + frontend refactor

- 1.3 KSeF dual dateType query
- 1.4 KSeF gate jako blocker
- 1.19 VCR integration tests dla KSeF
- 1.10 SSE endpoint
- 1.11 Per-step UI
- 1.12 Tryb expert (per-step buttons)
- 1.13 Usuń duplikaty (STEP\_EST\_MS, runDryRun, fake animation)
- 1.14 Ujednolicić numerację kroków
- 1.6 Webhook Service Bus (lub queue Tables)

### Tydzień 4 — multi-user start

- 2.1 Lock per (year, month)
- 2.2 Tabele runs + run\_steps
- 2.3 GET /close/runs z user info
- 2.4 GET /close/running
- 2.7 Audit log: started\_by
- 2.12 UI banner aktywnego lock'u

### Tydzień 5 — UX dla nie-Piotra

- 2.8 Tooltips, mniej żargonu, lepsze błędy
- 2.9 Onboarding screen



- 2.5 + 2.6 Vendor config UI
- 2.10 Email „paczka gotowa” do księgowej
- 2.11 Email „pipeline padł” do admina
- 2.13 E2E test równoległych sesji

## Tydzień 6 — higiena

- 3.1 + 3.2 Per-module coverage
- 3.3 + 3.4 Mutation na więcej modułów + PR gate
- 3.5 release-please
- 3.7 Property tests
- 3.8 + 3.9 E2E pełny UI flow + refresh scenario
- 3.10–3.14 Frontend cleanup
- 3.15 Performance testy
- 3.6 Plan strict pyright

## 9. Ryzyka i warunki powodzenia

### Główne ryzyka

#### Refactor orchestratora rozjedzie się z istniejącymi testami

Test\_orchestrator.py (572 LOC) prawdopodobnie testuje wewnętrzne metody \_step\_X. Po refaktorze na Step pattern testy się zepsują wszystkie. Mitygacja: przed refaktorem napisać testy behawioralne (black-box) opisujące oczekiwane zachowanie. Dopiero potem refaktor.

#### Migracja checkpointu z blob na Azure Tables może zepsuć historię

Stary mechanizm checkpoint w blobie + history JSONL. Nowy w Tables. Mitygacja: okres przejściowy — czytaj z obu, zapisuj do nowego. Po 2-3 miesiącach stabilności wywalić stary kod.

#### Lock per (year, month) może zablokować legitime parallel runs

Jeśli ktoś chce odpalić maj i czerwiec równoległe — działa. Ale gdy lock wygaśnie podczas legitime długiego runu — drugi user przejmie. Mitygacja: heartbeat co 30s odświeża lock; pipeline ma timeout 60min; przekroczenie → notification do admina.

#### Księgowa zacznie używać tool'a przed fazą 2 i utrwali złe wzorce

Jeśli wpuścisz drugiego usera już teraz, trafi na obecne bugi (checkpoint, refresh, destruktywny inbox). Mitygacja: NIE wpuszczać innych userów przed końcem fazy 1. Faza 2 wprowadza ich dopiero gdy fundament jest stabilny.

#### Wybór kolejki zdarzeń (Service Bus vs Tables) jest binarny i ma konsekwencje

Service Bus = profesjonalne queue, koszt ~5 zł/mies, complex. Tables jako pseudo-queue = darmowe (już płacisz), prostsze, ale ograniczone. Rekomendacja: dla single-tenant HUMIO Tables wystarczą. Service Bus tylko gdy faktycznie potrzebujesz dead-letter queue i retry policy.

### Warunki powodzenia

- Nie pomijać tygodnia 1. Sieć bezpieczeństwa (testy) musi być przed refaktorem.
- Po każdej fazie 2 tygodnie obserwacji. Pipeline w prawdziwych warunkach, nie spieszyć się z następną fazą.
- Każdy nowy Step od początku z pełnym pyright strict i 90%+ coverage. Nie powtarzać błędów basic mode.
- Frontend: każdy nowy komponent + każdy zmieniany komponent ma vitest.
- Mutation score monitorować po każdej fazie. Spadek = brakuje testów.
- Dokumentacja techniczna na bieżąco. CLAUDE.md, ADR-y (Architecture Decision Records) dla każdej dużej decyzji (Step pattern, SSE, Tables vs Service Bus).

## 10. Podsumowanie

Projekt `zdrovena-reconciliation` jest pod kątem inżynierskim lepiej zbudowany niż 90% projektów wewnętrznych w średniej wielkości firmach. Architektura jest świadoma, infrastruktura solidna, jakość testów ponadprzeciętna. Problemy które dziś bolą (`checkpoint`, `inbox`, `refresh`, `KSeF` dla księgowej) wynikają z jednego źródła: produkt został zbudowany z perspektywy CLI `single-user`, a teraz jest używany jak produkt webowy z wieloma użytkownikami w przyszłości.

Plan refaktoru w trzech fazach (łącznie 5-6 tygodni roboty programistycznej) przekształca to w produkt który:

- Działa prawidłowo dla obecnego użytkownika (Piotr) — koniec fałszywego progresu, `checkpoint`u który kłamie, znikania widoku po `refresh`.
- Otwiera się bezpiecznie dla nowych użytkowników w HUMIO — księgowej, współników — z `lock`'iem chroniącym przed `race conditions` i UX-em dla nie-technicznych użytkowników.
- Zachowuje wszystkie dotychczasowe zalety — `Step pattern` wzmacnia `mutation testing` i `fitness functions`, nie osłabia. `Azure Tables` są naturalnym rozszerzeniem dzisiejszej infrastruktury, nie wymianą.
- Ma sieć bezpieczeństwa do dalszego rozwoju — `vitest` dla frontu, `property tests` dla backendu, `per-module coverage`, `release-please` dla wersjonowania.

### Trzy najważniejsze rzeczy do zrobienia w tym tygodniu

1. Shopify `HMAC fail-closed` (15 minut, zamyka krytyczny wektor bezpieczeństwa).
2. Dodać `vitest` do frontendu i napisać pierwsze 5 testów dla `normalizeStepKey`, `formatters`, `markRunningAsError`. Bez tego refactor frontu w tygodniu 3 będzie ślepy.
3. Zacząć pisać behawioralne testy orchestratora opisujące jak ma działać po refaktorze (nie jak działa dziś). To Twoja umowa z przyszłym sobą.

---

#### Koniec raportu.

Raport wygenerowany na podstawie analizy repozytorium `github.com/PiotrGry/zdrovena-reconciliation` w wersji 2.6.0 (czerwiec 2026). Przegląd objął cały backend Python, frontend React, infrastrukturę Terraform, workflowy CI/CD oraz pakiet testów. Wszystkie cytowane lokalizacje kodu są precyzyjne i można je zweryfikować bezpośrednio w repo.